

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3069 – Computación Paralela y Distribuida

Marlón Fuentes

Sección: 40



Hoja de Trabajo 2

Mario Fernando Rocha – 23501

Guatemala 20 de septiembre de 2026

Parte 1.

1. Qué significa Rank y size?

El Size es el número total de procesos que se lanzaron dentro de un comunicador. Este es fijo durante toda la ejecución. (*MPI_COMM_WORLD* es el que agrupa a todos por defecto).

El rank es identificador único dentro de ese comunicador, un entero entre 0 y size-1. No es un ID de hilo o de proceso, es una etiqueta lógica que MPI asigna al lanzar los procesos, y es el que cada proceso usa para saber “quién soy yo” y diferenciarse del resto (por eso el mismo binario corriendo N copias, puede comportarse distinto según su rank – como aquí, donde solo el comunicador se identifica como rank 0).

2. Qué pasa con 1 núcleo?

Al ejecutar el programa con un solo proceso (*mpirun -np 1*), el valor de size es 1 y únicamente existe el rank 0, por lo que la salida se reduce a una sola línea: *Rank 0 comunicador*. No se produce ningún error, ya que MPI no impone un número mínimo de procesos distintos de uno.

Parte 2.

1. Qué sucede antes y después de la barrera?

Antes de llegar a la barrera, cada proceso avanza de forma independiente y a su propio ritmo; en este programa, cada rank simula tiempos de trabajo distintos mediante *sleep(rank % 3)*. Como consecuencia, el orden en que los procesos imprimen sus mensajes de la Etapa 1 es no determinista: depende de la planificación del sistema operativo y no sigue necesariamente el orden numérico de los ranks.

La siguiente salida, obtenida al ejecutar el programa con 4 procesos, ilustra este comportamiento:

```
Rank 3 con size 4
mario-rocha@Marios-MacBook-Air HDT2 % mpicc -o parte2 parte2.c && mpirun -np 4 ./parte2
Rank 1: [Etapa 1] iniciando trabajo previo...
Rank 3: [Etapa 1] iniciando trabajo previo...
Rank 3: [Etapa 1] trabajo previo terminado, esperando en la barrera
Rank 2: [Etapa 1] iniciando trabajo previo...
Rank 0: [Etapa 1] iniciando trabajo previo...
Rank 0: [Etapa 1] trabajo previo terminado, esperando en la barrera
Rank 1: [Etapa 1] trabajo previo terminado, esperando en la barrera
Rank 2: [Etapa 1] trabajo previo terminado, esperando en la barrera
Rank 2: [Etapa 2] todos sincronizados, continuando con la siguiente fase
Rank 0: [Etapa 2] todos sincronizados, continuando con la siguiente fase
Rank 1: [Etapa 2] todos sincronizados, continuando con la siguiente fase
Rank 3: [Etapa 2] todos sincronizados, continuando con la siguiente fase
```

2. Para qué sirve MPI_BARRIER?

Es un punto de sincronización colectivo: cada proceso que lo llama se bloquea ahí hasta que todo los procesos del comunicador también la hayan llamado. No sincroniza datos (no transmite nada, a

diferencia de MPI_Bcast), solo sincroniza tiempo de ejecución – garantiza que ninguna etapa posterior empiece mientras algún proceso siga en la etapa anterior. Se usa típicamente para medir tiempos de forma justa o para asegurar que un recurso compartido esté completo antes de que otros procesos lo lean.

Parte 3.

1. Qué hace cada instrucción?

MPI_Bcast: Operación colectiva – la deben llamar todos los procesos del comunicador con la misma firma. El proceso root (el rank 0 para el ejercicio) hace de emisor: lo que tiene en su buffer se copia al buffer de todos lo demás. No es un *send/recv* manual entre pares; es una sola instrucción que internamente MPI implementa de forma eficiente (típicamente como árbol de difusión, no uno-a-uno secuencial).

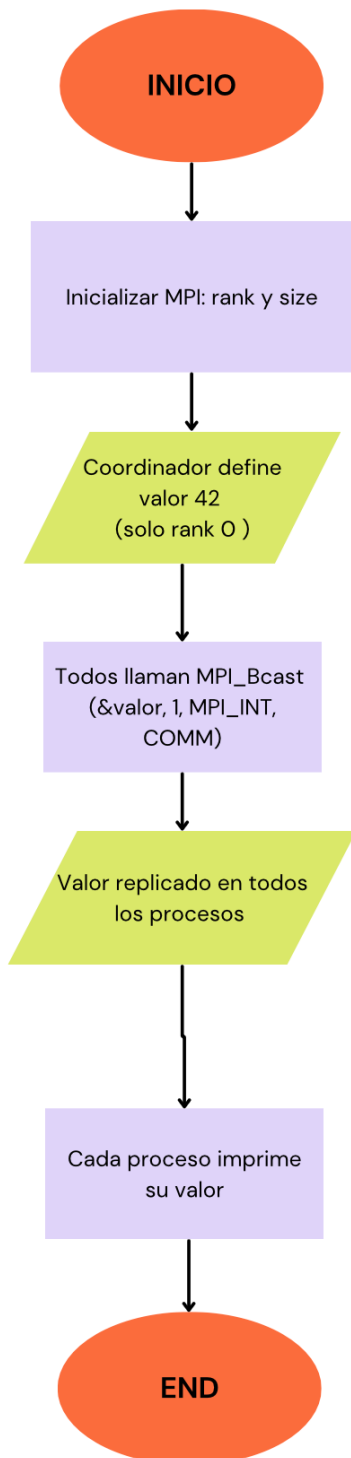
MPI_INT: es un MPI_Datatype, una constante que le dice a MPI cómo interpretar los bytes curdo que se están moviendo – en este caso, como un *int* de C. MPI necesita este dato porque trabaja a nivel de bytes/red y debe saber el tamaño y layout del dato para copiarlo/transmitirlo correctamente.

2. Qué sucede con 99999?

El comportamiento no cambia: el valor se difunde correctamente a todos los procesos sin generar ningún error. La razón es que MPI_INT corresponde a un entero de 32 bits con signo, cuyo rango llega aproximadamente 2,147,483,647; 99999 se encuentra muy por debajo de ese límite, por lo que no existe riesgo de desbordamiento.

Este resultado sería distinto si se utilizara un tipo de dato más pequeño, como MPI_SHORT (entero de 16 bits, con un máximo de 32,767): en ese caso, el valor 99999 excedería el rango representable y produciría un desbordamiento, generando un valor incorrecto en los procesos receptores. Este contraste es relevante porque demuestra que MPI_Bcast en sí no impone restricciones sobre la magnitud del dato transmitido; la limitación proviene exclusivamente del tipo de dato (MPI_Datatype) especificado en la llamada.

Diagrama



Link del repositorio:

https://github.com/rocha30/HDT2_Paralela.git